

EasyVisa — Project-Based Interview Pack

Target: Full-Stack Developer, 5 years (Angular · Grails/Groovy · PostgreSQL · Neo4j · Angular Formly)

Idea: Every answer is anchored in **real work I did on EasyVisa** — so when an interviewer asks "where did you actually use X?", I have a concrete feature, pattern, and story instead of a textbook definition.

How to use this pack

- Each file = one interview area. Read the **Question** → **Answer** → **"In EasyVisa"** structure.
- The **"In EasyVisa"** block is the talking point — memorize the *flow and the decision*, not the code character-for-character.
- Say the answers **out loud**. Interviewers fail me on "project understanding," so the goal is: no matter what they ask, I bring it back to a real EasyVisa feature — the document portal, the Formly forms, the image cropper, the Postgres/Neo4j split.
- The golden rule: **every technical answer ends with "In EasyVisa, I..."** That single habit is what turns a generic candidate into someone who obviously built the thing.

The sections

#	File	Covers
0	<code>00-ONE-PAGE-CHEATSHEET.md</code>	20 must-know facts + one-liner answers for a 30-min pre-interview glance
1	<code>01-frontend.md</code>	Angular deep dive — lifecycle hooks, component communication, RxJS (Observable/Subject/BehaviorSubject), memory leaks/ <code>takeUntil</code> /async pipe, <code>switchMap</code> /debounce, the document-upload progress code, the image cropper, reusable components, change detection
2	<code>02-forms-deep-dive.md</code>	The signature file — Angular Formly JSON-driven forms, <code>fieldGroup</code> nesting, <code>fieldArray</code> repeatable sections, conditional show/hide, dynamic validation, nested reactive-form state, validation bubbling, reactive vs template-driven
3	<code>03-backend-grails-groovy.md</code>	Grails/Groovy for someone who used it and is rusty — what Grails is, GORM, controllers, REST endpoints, how the Angular app consumed them (honest: I was frontend-focused)
4	<code>04-database-postgres-neo4j.md</code>	Why two databases — Postgres for relational/transactional, Neo4j for graph relationships, nodes/relationships, basic Cypher, graph vs relational, "who's connected to this case"
5	<code>05-behavioral-and-stars.md</code>	60-sec pitch, my role, architecture to draw, 3–4 STAR stories, "what I'd do differently," collaboration/Agile

Key project facts (know these cold)

- **EasyVisa** — visa **case management platform for immigration attorneys**. Manage applications, documents, and the relationships between cases, applicants, and petitioners. **Nov 2021 – Jan 2024**.
- **Frontend** — **Angular** (I owned this). NgModule architecture, RxJS-heavy, reactive forms, **Angular Formly** for dynamic forms.
- **Backend** — **Grails / Groovy** (convention-over-configuration, built on Spring + Hibernate). Exposed REST endpoints I consumed.
- **Databases** — **PostgreSQL** for relational/transactional data + **Neo4j** graph DB for the relationships between applicants ↔ petitioners ↔ cases.
- **My modules** — the full **document portal** (accordion upload panels, `HttpClient` progress uploads), **JSON-driven dynamic/nested/repeatable Formly forms with conditional logic** (my signature achievement), a **customized image cropper** for applicant photos, **reusable components**, an **Angular version upgrade**, and **unit tests** (Jasmine/Karma) for all UI components/services + specs/user manuals.
- **Team** — ~6–8 (frontend, Grails/Groovy backend devs, QA, lead). I was the frontend engineer owning the document portal and dynamic forms.
- **The one thing to remember** — if I only get to say one thing, it's the **JSON-driven Formly forms**: visa forms are huge, conditional, and repeatable, and I made them *configuration instead of code*.

Honesty note (important — this is what makes it credible)

I was a **frontend engineer**. I owned the Angular side end-to-end, and I **consumed** the Grails/Groovy REST APIs — I did not write the backend day to day, and Neo4j/Cypher was primarily server-side. So my backend and graph-DB answers are framed as "*here's how it worked, here's how I consumed it, and here's what I understand about why it was built that way*" — not as a Grails or Neo4j expert. Interviewers respect a confident "I owned the frontend, I understood the whole system, and here's the boundary of what I did myself" far more than someone who overclaims and then can't go deep. Where I'd improve the system (chunked uploads, backend-served form JSON), I say so — owning trade-offs reads as senior.

EasyVisa — One-Page Cheat Sheet

30-minute pre-interview glance. 20 must-know facts, each with a one-liner I can say out loud.

The pitch (memorize this verbatim)

"EasyVisa is a visa case management platform for immigration attorneys — managing applications, documents, and the relationships between cases, applicants, and petitioners. I was the frontend engineer on an Angular app talking to a Grails/Groovy backend, with PostgreSQL for transactional data and Neo4j for the graph of relationships. I owned the document portal and the dynamic Formly forms — I made huge, conditional, repeatable visa forms into JSON configuration instead of hand-coded components."

The 20 facts

1. **What it is** — visa case management for immigration attorneys. Cases, applicants, petitioners, documents. Nov 2021 – Jan 2024.
2. **My role** — frontend engineer, team of ~6–8. I owned the **document portal** and **dynamic forms**, and built shared components.
3. **Stack in one breath** — Angular front end, Grails/Groovy back end, PostgreSQL + Neo4j.
4. **My signature achievement** — **JSON-driven, nested, repeatable Angular Formly forms with conditional logic**. "New forms became config, not code."
5. **Why Formly** — visa forms are huge, conditional, and repeatable (N applicants, each with nested address/employment history). Hand-coding is unmaintainable; Formly renders from a JSON field config.
6. **Formly nesting** — `fieldGroup` for nested groups, `fieldArray` for repeatable sections (add/remove N applicants).
7. **Formly conditional logic** — `expressions: { hide: 'model.hasEmployer === false' }` for show/hide; dynamic `required` the same way.
8. **Formly validation** — nested `FormGroup` aggregation + custom validators; errors bubble up so the **submit button disables** until the whole nested tree is valid.
9. **Reactive vs template forms** — I used **reactive forms** (form model is the source of truth), which Formly is built on. Template-driven doesn't scale to this complexity.
10. **Document portal** — accordion panels by document type; each panel has its own upload/progress/error state so one failure doesn't sink the others.
11. **Large uploads** — `HttpClient` with `reportProgress: true, observe: 'events'`, read `HttpEventType.UploadProgress` for a per-file progress bar. Next step would've been chunked/resumable.
12. **Image cropper** — customized an Angular image-cropper library to enforce aspect ratio/output size/format for applicant photos, then fed the output blob into the upload flow.
13. **Memory leaks** — `takeUntil(destroy$)` completed in `ngOnDestroy`, plus the **async pipe** in templates so subscriptions clean themselves up.

14. **Duplicate/rapid API calls** — `debounceTime` + `switchMap` so only the latest request wins and stale ones are cancelled (typeaheads, live validation).
15. **State** — feature services wrapping the Grails REST endpoints; **BehaviorSubject** for shared state; the Formly **model object is the single source of truth** for form state (made draft save/restore trivial).
16. **Component communication** — `@Input` / `@Output` for parent-child; shared service + `BehaviorSubject` for cross-tree.
17. **Backend** — **Grails**: Groovy-based, convention-over-configuration, built on Spring + Hibernate; **GORM** for persistence; controllers expose REST JSON. I **consumed** these APIs (I was frontend-focused).
18. **Why two databases** — **Postgres** = relational/transactional (case data, integrity, ACID). **Neo4j** = the graph of who-is-connected-to-whom (applicant ↔ petitioner ↔ case ↔ employer/family).
19. **Graph vs SQL** — "show everyone connected to this case, 3 hops out" is a natural Cypher traversal and a painful pile of self-joins in SQL. Graph DBs store relationships as first-class edges.
20. **Also did** — drove an **Angular version upgrade** (incremental `ng update`, fix breaking changes at each step), wrote **Jasmine/Karma unit tests** for all components/services, wrote **specs and user manuals**.

Rapid-fire one-liners (if pushed)

They ask...	I say...
Lifecycle hooks	" <code>ngOnInit</code> to load, <code>ngOnChanges</code> for input changes, <code>ngOnDestroy</code> to complete my <code>destroy\$</code> — I used all three in the upload panels."
Observable vs Subject vs BehaviorSubject	"Observable is a stream; Subject is a multicast stream you push into; BehaviorSubject remembers the last value — I used it for shared form/case state."
Prevent memory leaks	" <code>takeUntil(destroy\$)</code> + async pipe — standard in every EasyVisa component."
Restrict multiple API calls	" <code>debounceTime</code> + <code>switchMap</code> — latest wins, stale cancelled."
Route guards / lazy loading	" <code>CanActivate</code> guarded attorney routes; feature modules lazy-loaded; resolver pre-fetched case data before the page opened."
Global exception handling	"Per-call <code>catchError</code> → user message, plus a shared HTTP interceptor for auth + global failures."
Interface vs abstract class	"Interface = a contract, no implementation, multiple; abstract class = partial implementation + shared state, single. I typed my Formly configs and API models with TS interfaces."
try/catch/finally	" <code>try</code> risky code, <code>catch</code> handle, <code>finally</code> always runs (cleanup). In RxJS the equivalent is <code>catchError</code> + <code>finalize</code> ."
Why Neo4j	"Relationships are the domain. Traversing 'who's connected to this case' is native in a graph, painful in SQL joins."
Reactive vs template forms	"Reactive — model is source of truth, testable, scales to nested/dynamic. Formly is built on reactive forms."

Confidence anchors (say these, they're all true)

- "I owned the frontend end-to-end and I understood the whole system."
- "I was frontend-focused — I consumed the Grails REST APIs; Neo4j was mostly server-side, but I understand why we had it."
- "My proudest work is turning unmaintainable hand-coded visa forms into JSON-driven Formly config."

1. Front-End (Angular) — Interview Q&A tied to EasyVisa

Stack: Angular (NgModule architecture), RxJS-heavy, reactive forms + **Angular Formly** for dynamic forms, `HttpClient` against a Grails/Groovy REST backend. I owned the **document portal** and the **dynamic forms**, built reusable components, customized an image cropper, wrote Jasmine/Karma unit tests, and drove an Angular version upgrade.

Golden rule for every answer below: define it cleanly, then land it with **"In EasyVisa, I..."**.

Q1. Explain Angular lifecycle hooks — which did you actually use?

Answer. Lifecycle hooks let me run code at defined moments in a component's life:

- `ngOnChanges(changes)` — runs when an `@Input` reference changes (before `ngOnInit` and on every input update).
- `ngOnInit()` — once, after the first inputs are set. Where I do initial data loads and form setup.
- `ngAfterViewInit()` — after the component's view (and `@ViewChild` refs) are ready. Needed when I touch a child component/DOM element directly.
- `ngOnDestroy()` — right before Angular tears the component down. Where I clean up subscriptions.

In EasyVisa. In the document portal upload panels I used all the key ones concretely: - `ngOnInit` to build the Formly config and fetch the already-uploaded documents for the case. - `ngOnChanges` because each accordion panel took the document-type config as an `@Input` — when the selected case changed, `ngOnChanges` reset the panel's state. - `ngAfterViewInit` for the **image cropper** — I needed the cropper's `@ViewChild` to exist before wiring the crop output. - `ngOnDestroy` to `next()` + `complete()` my `destroy$` subject so in-flight upload progress subscriptions didn't leak.

```
export class DocumentUploadPanelComponent implements OnInit, OnChanges, OnDestroy {
  @Input() documentType!: DocumentTypeConfig;
  private destroy$ = new Subject<void>();

  ngOnInit(): void { this.loadExistingDocuments(); }
  ngOnChanges(): void { this.resetPanelState(); } // case/type input changed
  ngOnDestroy(): void { this.destroy$.next(); this.destroy$.complete(); }
}
```

Q2. How do components communicate in your app?

Answer. Three mechanisms depending on the relationship: - **Parent** → **child**: `@Input()`. - **Child** → **parent**: `@Output()` `EventEmitter`. - **Siblings / across the tree**: a shared service holding a `BehaviorSubject`.

In EasyVisa. The document portal was a parent "case documents" component containing many accordion **upload-panel** children. The parent passed each panel its config via `@Input`, and each panel emitted back up when a file finished uploading so the parent could update the case's overall completeness indicator:

```
// upload-panel.component.ts
@Input() documentType!: DocumentTypeConfig;
@Input() caseId!: string;
@Output() documentUploaded = new EventEmitter<UploadedDocument>();

onUploadComplete(doc: UploadedDocument): void { this.documentUploaded.emit(doc); }
```

```
<app-upload-panel *ngFor="let type of documentTypes; trackBy: trackByType"
                  [documentType]="type" [caseId]="caseId"
                  (documentUploaded)="onDocumentUploaded($event)">
</app-upload-panel>
```

For state that many unrelated components needed — the currently selected case, the logged-in attorney — I used a **shared service** + `BehaviorSubject` rather than threading inputs through every level.

Q3. Observable vs Subject vs BehaviorSubject — explain the difference and where you used each.

Answer. - `Observable` — a lazy stream you subscribe to; it produces values. It's *unicast* by default (each subscriber gets its own execution). This is what `HttpClient` and `valueChanges` return. - `Subject` — both an `Observable` and an `Observer`. It's **multicast** — I can `.next()` values into it and every subscriber gets them. But a late subscriber misses values emitted before it subscribed. - `BehaviorSubject` — a `Subject` that **remembers the last value** and replays it immediately to any new subscriber. It requires an initial value. Perfect for "current state."

In EasyVisa. - **Observable** everywhere I called the Grails APIs (`this.http.get<Case>(...)`) and on reactive form `valueChanges`. - **Subject** for my `destroy$` unsubscribe signal and for "trigger this action" streams (e.g. an upload-request subject feeding a debounced pipe). - **BehaviorSubject** for shared current state — the selected case and the current attorney/session — exposed read-only via `asObservable()` so consumers could read the latest value but only the service could push:

```
@Injectable({ providedIn: 'root' })
export class CaseContextService {
  private selectedCase$ = new BehaviorSubject<VisaCase | null>(null);
  readonly selectedCase = this.selectedCase$.asObservable(); // read-only out
  setSelectedCase(c: VisaCase): void { this.selectedCase$.next(c); }
}
```

Follow-up: "Why BehaviorSubject over Subject there?" Because a component that loads *after* the case was selected still needs the current case immediately — `BehaviorSubject` replays it; a plain `Subject` would leave that component with nothing until the next change.

Q4. How do you prevent memory leaks in Angular? (the #1 question)

Answer. Leaks in Angular almost always come from **RxJS subscriptions that outlive the component**. If I `subscribe()` and never clean up, the stream keeps a reference to the component so it's never garbage-collected, and its callback keeps firing after the view is gone. Three tools, in order of preference:

1. **async pipe** in the template — Angular subscribes and unsubscribes for me.
2. `takeUntil(destroy$)` for imperative subscriptions — a `Subject` I complete in `ngOnDestroy`.
3. Manual `Subscription.unsubscribe()` only for one-offs.

In EasyVisa. `takeUntil(destroy$)` was my standard in every component, and it mattered most in the upload panels because an upload progress stream is long-lived — if the attorney navigated away mid-upload, I did **not** want a dead component still receiving progress events:

```
this.uploadService.upload(file, this.caseId)
  .pipe(takeUntil(this.destroy$))
  .subscribe(event => this.handleProgress(event));

ngOnDestroy(): void { this.destroy$.next(); this.destroy$.complete(); }
```

In templates I leaned on the **async pipe** so those subscriptions never needed manual cleanup:

```
<div *ngIf="selectedCase$ | async as case">{{ case.applicantName }}</div>
```

Follow-up: "Anything newer?" In modern Angular you can drop the boilerplate subject with `takeUntilDestroyed()` from `@angular/core/rxjs-interop` — that's the direction I'd move the codebase.

Q5. How do you restrict multiple / duplicate API calls? (switchMap, debounce)

Answer. Two problems, two tools: - **Too many calls from rapid input** (typing in a search/validation field) → `debounceTime` to wait for a pause before firing. - **Stale responses arriving out of order / redundant in-flight calls** → `switchMap`, which cancels the previous inner request when a new value arrives, so **only the latest wins**.

Together they're the canonical "search-as-you-type" pattern, and they also stop double-submits.

In EasyVisa. Some visa form fields validated against the backend (e.g. checking an identifier/reference number). Firing on every keystroke would hammer the Grails API and could show a stale result. So:

```
this.referenceControl.valueChanges.pipe(
  debounceTime(400),           // wait for the user to stop typing
  distinctUntilChanged(),      // ignore no-op changes
  switchMap(value =>           // cancel the previous validation call
    this.caseService.validateReference(value).pipe(
      catchError(() => of({ valid: false }))
    )
  ),
),
```

```
takeUntil(this.destroy$)
).subscribe(result => this.referenceStatus = result);
```

I also used `switchMap` when the selected case changed — switching the "load this case's documents" call so an old, slow case-load couldn't overwrite the newly selected case's data.

Follow-up: `switchMap` vs `mergeMap` vs `concatMap`? `switchMap` cancels the previous (latest wins — search, validation, selection). `mergeMap` runs all in parallel (independent uploads). `concatMap` queues them in order (when order matters). For **document uploads** I actually wanted a throttled-parallel behavior, not `switchMap` — I never want a second file to *cancel* the first.

Q6. Walk me through the large document-upload code.

Answer. Attorneys upload many documents per case. Uploading naively blocks the UI and risks timeouts, and I need a **per-file progress bar** and isolated error handling. The key is `HttpClient` with `reportProgress: true` and `observe: 'events'`, then reading the event stream:

```
uploadDocument(file: File, caseId: string): Observable<UploadState> {
  const formData = new FormData();
  formData.append('file', file);
  formData.append('caseId', caseId);

  return this.http.post('/api/documents', formData, {
    reportProgress: true,
    observe: 'events'
  }).pipe(
    map((event: HttpEvent<any>) => {
      switch (event.type) {
        case HttpEventType.UploadProgress:
          return { status: 'uploading',
                    progress: Math.round(100 * event.loaded / (event.total ?? 1)) };
        case HttpEventType.Response:
          return { status: 'done', document: event.body };
        default:
          return { status: 'pending', progress: 0 };
      }
    }),
    catchError(err => of({ status: 'error', message: this.toUserMessage(err) })),
  );
}
```

In EasyVisa. Each accordion panel (grouped by document type) subscribed to this with `takeUntil(destroy$)` and rendered its own progress bar. Crucially, **each panel had its own progress/error state**, so if one file failed, that panel showed the error while the others kept going — one bad upload didn't sink the case. Uploads were concurrent but throttled so I didn't fire 20 requests at once.

Follow-up: "What would you improve?" For very large files the right move is **chunked/resumable upload** — split the file, upload chunks, resume on failure. I flagged that as the next iteration; we shipped the progress-stream version first because it covered the real file sizes attorneys used.

Q7. You customized an image cropper — why and how?

Answer. Certain visa documents — applicant photos in particular — must meet strict dimension, aspect-ratio, and format requirements. I didn't want attorneys pre-editing images in another tool, so I wrapped and customized an **Angular image-cropper** library to enforce those rules in-app.

In EasyVisa. I configured the cropper to lock the aspect ratio and output size, then **post-processed the output blob** before it entered the upload flow — so what got cropped was exactly what got uploaded, already compliant:

```
// wired the cropper's output into the same upload pipeline as Q6
onImageCropped(event: ImageCroppedEvent): void {
  const croppedBlob = event.blob!; // enforced aspect + size
  const file = new File([croppedBlob], 'applicant-photo.png', { type: 'image/png' });
  this.uploadService.uploadDocument(file, this.caseId)
    .pipe(takeUntil(this.destroy$))
    .subscribe(state => this.handleUpload(state));
}
```

The customization was mostly (1) constraining the crop box to the required aspect ratio, (2) forcing the output dimensions/format, and (3) feeding the resulting blob straight into the document upload. Wrapping it in my own component also made it a **reusable** control other parts of the app could drop in.

Q8. You built reusable components — give an example and the benefit.

Answer. I extracted anything used in more than one place into shared, configuration-driven components. The benefit is consistency and speed — a new use case becomes a config change, and a bug gets fixed once.

In EasyVisa. My reusable pieces were: - The **upload-panel** component (config: document type, accept rules, required flag). - Custom **Formly field types** (see [02-forms-deep-dive.md](#)) — the biggest reuse win, because a whole new form section became JSON, not new code. - The **image-cropper** wrapper. - Common form controls and layout wrappers.

```
// a shared, configurable upload panel — reused per document type across the app
@Component({ selector: 'app-upload-panel', templateUrl: './upload-panel.component.html' })
export class UploadPanelComponent {
  @Input() documentType!: DocumentTypeConfig; // label, accept, required, multiple
  @Input() caseId!: string;
  @Output() documentUploaded = new EventEmitter<UploadedDocument>();
}
```

The payoff: adding a new document type to the portal meant adding a config entry, not writing a new component — and every panel behaved identically (progress, errors, validation).

Q9. How does change detection work, and did you use OnPush?

Answer. Angular's default strategy dirty-checks the component tree whenever something *might* have changed — triggered by zone.js patching async APIs (events, timers, XHR).

`ChangeDetectionStrategy.OnPush` narrows that: the component is only checked when an `@Input` **reference** changes, an event fires inside it, or an `async`-pipable observable it uses emits.

In EasyVisa (honest). We ran mostly the **default** strategy. The document portal, with many panels and live upload progress, is exactly the kind of tree that benefits from **OnPush + immutable updates + the async pipe** — that's a concrete perf improvement I can speak to. Where I needed to force a refresh after something Angular didn't see (e.g. a callback from the cropper library), I injected `ChangeDetectorRef` and called `detectChanges()`. Being honest that we were default-strategy, and knowing exactly which components I'd move to OnPush and why, reads better than claiming we were fully optimized.

Q10. How did you handle state, APIs, and errors overall?

Answer. Feature services wrapping the Grails REST endpoints, Observables consumed via the **async pipe**, **reactive forms** as the source of truth for form state, and **BehaviorSubject** services for shared state. Errors were handled per-call with `catchError` mapping to user-friendly messages, plus a shared **HTTP interceptor** for auth headers and global failures.

In EasyVisa. For the forms specifically, I kept the **Formly model as the single state object**, which made **draft save/restore** trivial — I could serialize the whole nested model, store it, and rehydrate the form later. That's a nice architectural point: the form's data and its state were the same object.

```
// error handling per call, consistent user messaging
getCase(id: string): Observable<VisaCase> {
  return this.http.get<VisaCase>(`/api/cases/${id}`).pipe(
    catchError(err => { this.notifications.show(this.toUserMessage(err)); return EMPTY; })
  );
}
```

(Global exception handling / interceptor detail is covered in `05-behavioral-and-stars.md` and the forms/backend files.)

Q11. Route guards, lazy loading, resolvers — did you use them?

Answer. - **Guards** (`CanActivate`) decide whether a route can be entered — I used them so only authenticated attorneys reached case routes. - **Lazy loading** — feature modules loaded on demand (`LoadChildren`) so the initial bundle stayed small. - **Resolvers** pre-fetch data *before* a route activates, so the component never renders half-loaded.

In EasyVisa. Case detail and the document portal were **lazy-loaded feature modules**. A **resolver** pre-loaded the case (and its document metadata) before the portal opened, so the accordion rendered fully populated rather than flashing empty and then filling in:

```
{
  path: 'cases/:caseId',
  loadChildren: () => import('./case/case.module').then(m => m.CaseModule),
  canActivate: [AuthGuard],
  resolve: { case: CaseResolver }
}
```

Trade-off I'd mention: resolvers delay navigation until the data arrives, so I only resolved the *critical* above-the-fold case data and lazy-loaded the rest inside the component.

Q12. The Angular version upgrade — how did you approach it?

Answer. Incrementally and safely — never jump multiple majors at once. I used the official `ng update` path, going one major at a time, building and running the test suite at each step, and fixing breaking changes and deprecations as they surfaced. The Angular Update Guide told me exactly what changed per version.

In EasyVisa. I drove the upgrade: bumped one major, resolved the compilation/deprecation issues (RxJS operator imports, typing changes, deprecated APIs), got the Jasmine/Karma suite green, verified the document portal and Formly forms manually, then moved to the next major. Having **unit tests on the components and form logic** is exactly what made this safe — the tests caught regressions the compiler didn't.

Q13. How did you test the frontend?

Answer. Jasmine + Karma unit tests for components and services. I tested component logic, form/validation behavior, and service methods (mocking `HttpClient` with `HttpTestingController`). I aimed to cover the branching logic — especially the conditional form rules, where regressions hide.

In EasyVisa. I wrote unit tests for all my UI components and services, which is what made the version upgrade and ongoing refactors safe. The highest-value tests were around the **Formly conditional logic** — asserting that a field hid/showed and that `required` toggled correctly as the model changed — because that logic is easy to break silently. I also documented specs and user manuals so other devs could adopt the reusable pieces.

Rapid-fire recap

Topic	EasyVisa pattern
Lifecycle hooks	<code>ngOnInit</code> load, <code>ngOnChanges</code> reset panel on input change, <code>ngAfterViewInit</code> for cropper, <code>ngOnDestroy</code> complete <code>destroy\$</code>
Component comms	<code>@Input</code> / <code>@Output</code> (portal ↔ panels), BehaviorSubject service (selected case)
Observable/Subject/BehaviorSubject	HTTP/ <code>valueChanges</code> / <code>destroy\$</code> + triggers / current case + session

Topic	EasyVisa pattern
Memory leaks	<code>takeUntil(destroy\$)</code> + async pipe
Restrict API calls	<code>debounceTime</code> + <code>switchMap</code> (latest wins)
Uploads	<code>HttpClient</code> <code>reportProgress</code> + <code>observe: 'events'</code> , per-panel state
Image cropper	customized lib, enforced aspect/size, blob → upload pipeline
Reusable components	upload-panel, custom Formly types, cropper wrapper
Change detection	default (know OnPush plan), <code>detectChanges()</code> after lib callbacks
Guards/lazy/resolver	<code>CanActivate</code> on case routes, lazy feature modules, <code>CaseResolver</code>
Upgrade	incremental <code>ng update</code> , tests green each step
Testing	Jasmine/Karma, focus on conditional form logic

2. Forms Deep Dive (Angular Formly) — the EasyVisa signature file

This is my strongest story. If I get one deep-dive question, I want it to be this one.

Stack: Angular **reactive forms** as the foundation, **Angular Formly** for JSON-driven dynamic forms, `fieldGroup` for nesting, `fieldArray` for repeatable sections, `expressions` for conditional logic and dynamic validation, custom Formly field types for reuse.

The headline: *Visa forms are huge, conditional, and repeatable. Hand-coding each one is unmaintainable. I made forms into JSON configuration instead of code — new forms and new conditional rules became data, not new components.*

Q1. What is Angular Formly and why did you choose it?

Answer. Formly is a library that **renders Angular reactive forms from a JSON configuration** instead of hand-written template markup. You describe the fields — key, type, label, validators, conditional rules — as an array of `FormlyFieldConfig` objects, and Formly builds the underlying `FormGroup` and the UI for you.

I chose it because **visa forms are the worst case for hand-coding**: - They're **huge** — dozens to hundreds of fields. - They're **conditional** — fields appear/disappear based on earlier answers ("Are you employed?" → show employer fields). - They're **repeatable** — a case can have N applicants, each with nested address and employment histories, each of which can itself repeat. - They **change** — immigration rules and form requirements shift, and I didn't want a code release for every field tweak.

Hand-coding that means a giant, brittle template per form and a new component every time. With Formly, a new form or a new rule is a **config change**.

The trade-off I'd volunteer: steeper initial learning curve and some complexity writing custom field types, versus huge long-term savings. At EasyVisa's scale of forms, it clearly paid off. If I only had two or three simple forms, plain reactive forms would've been simpler and Formly would've been over-engineering.

Q2. Reactive vs template-driven forms — which and why?

Answer. - **Template-driven** — form logic lives in the template with `ngModel`; Angular builds the model implicitly. Fine for small, simple forms. Hard to test, hard to do dynamic/cross-field logic. - **Reactive** — I build the form model explicitly in TypeScript (`FormGroup` / `FormControl`), so the **model is the source of truth**. It's typed, testable, synchronous to read, and supports dynamic controls and cross-field validation.

In EasyVisa. Reactive, without question — and Formly itself is built on reactive forms, so under the hood every Formly form is a `FormGroup`. Template-driven would never have handled nested, repeatable,

conditional visa forms. Because the model was explicit and central, I could serialize the whole form to save a **draft** and rehydrate it later — the form's data and its state were one object.

Q3. Show me a JSON-driven Formly form. How does the config map to UI?

Answer. Each field is a `FormlyFieldConfig`: `key` (where it lives in the model), `type` (which field component renders it), `props` (label, placeholder, required — formerly `templateOptions`), plus `validators` and `expressions`.

In EasyVisa — a simplified applicant section:

```
const applicantFields: FormlyFieldConfig[] = [
  { key: 'firstName', type: 'input',
    props: { label: 'First name', required: true } },
  { key: 'dateOfBirth', type: 'datepicker',
    props: { label: 'Date of birth', required: true } },
  { key: 'hasEmployer', type: 'checkbox',
    props: { label: 'Currently employed?' } },
  { key: 'employerName', type: 'input',
    props: { label: 'Employer name' },
    expressions: { hide: 'model.hasEmployer === false' } }, // conditional visibility
];
```

```
<form [formGroup]="form">
  <formly-form [form]="form" [fields]="applicantFields" [model]="model"></formly-form>
  <button [disabled]="form.invalid">Submit</button>
</form>
```

The `model` object is the form data; `form` is the underlying `FormGroup` Formly populates. Change the config array and the UI changes — no template edits.

Q4. How did you nest forms? Explain `fieldGroup`.

Answer. `fieldGroup` creates a **nested group of fields** — it maps to a nested `FormGroup`. If I give it a `key`, the nested fields live under that key in the model; without a key it's a purely visual grouping.

In EasyVisa. An applicant had a nested **address** and **employment** structure, so I nested groups:

```
{
  key: 'address', // nested FormGroup → model.address.*
  fieldGroup: [
    { key: 'line1', type: 'input', props: { label: 'Street', required: true } },
    { key: 'city', type: 'input', props: { label: 'City', required: true } },
    { key: 'country', type: 'select', props: {
      label: 'Country', required: true, options: this.countryOptions } },
  ]
}
```

This produced `model.address.line1`, `model.address.city`, etc. — clean, nested state that mirrored the real domain shape. Nested groups also let validation aggregate naturally (see Q7).

Q5. Repeatable sections — explain `fieldArray`. This is the hard part.

Answer. `fieldArray` renders a **repeatable list of a field template** — it maps to a `FormArray`. Formly gives you add/remove for each entry, and each entry can itself be a `fieldGroup` with nested fields (and even nested `fieldArray`s). This is how I handled "N applicants, each with M previous addresses."

In EasyVisa — the standout structure, repeatable applicants each containing repeatable address history:

```
const fields: FormlyFieldConfig[] = [{
  key: 'applicants',
  type: 'repeat', // my custom repeating-section field type (Q8)
  fieldArray: {
    fieldGroup: [
      { key: 'firstName', type: 'input',
        props: { label: 'First name', required: true } },
      { key: 'hasEmployer', type: 'checkbox',
        props: { label: 'Currently employed?' } },
      { key: 'employerName', type: 'input',
        props: { label: 'Employer' },
        expressions: { hide: 'model.hasEmployer === false' } },
      {
        key: 'addressHistory',
        type: 'repeat', // nested repeatable INSIDE each applicant
        fieldArray: {
          fieldGroup: [
            { key: 'line1', type: 'input', props: { label: 'Street', required: true } },
            { key: 'fromDate', type: 'datepicker', props: { label: 'From', required:
true } },
            { key: 'toDate', type: 'datepicker', props: { label: 'To' } },
          ]
        }
      }
    ]
  }
}];
```

This yields `model.applicants[i].addressHistory[j].line1` — arbitrarily nested, all from JSON. The **hard parts** were: 1. Keeping the nested reactive-form **state valid as sections were added/removed** — Formly rebuilds the underlying `FormArray` / `FormGroup`, and I had to make sure validation and conditional expressions re-evaluated correctly on each add/remove. 2. **Performance** with many repeated groups — I kept field types lean and avoided heavy work in expressions. 3. Making the **custom repeat field type** reusable so every repeatable section (applicants, addresses, dependents) used the same add/remove UI.

Q6. How did the conditional show/hide and dynamic validation work?

Answer. Formly `expressions` — functions (or expression strings) evaluated against the current `model` / `field` that drive properties reactively. The common ones: - `expressions: { hide: '...' }` — show/

hide a field. - `expressions: { 'props.required': '...' }` — make a field conditionally required. - `expressions: { 'props.disabled': '...' }` — enable/disable.

When a hidden field is hidden, Formly also removes its control from validation, so a hidden "employer" field can't block submit.

In EasyVisa — conditional *and* dynamically required together:

```
{
  key: 'employerName',
  type: 'input',
  props: { label: 'Employer name' },
  expressions: {
    hide: 'model.hasEmployer === false',
    'props.required': 'model.hasEmployer === true',    // required only when shown
  }
}
```

Function form when the rule was more complex (cross-field):

```
{
  key: 'visaExpiryDate',
  type: 'datepicker',
  props: { label: 'Visa expiry' },
  expressions: {
    'props.required': (field) => field.model?.currentlyInCountry === true,
    hide: (field) => !field.model?.hasExistingVisa,
  }
}
```

This is the core of why Formly won: **conditional logic became configuration**. A new "show X when Y" rule was one line in JSON, not new component code and new tests.

Q7. How did validation bubble up so the submit button disabled correctly?

Answer. Because Formly builds a real reactive-forms tree, validity **aggregates automatically**: a `FormArray` is invalid if any entry is invalid, a `FormGroup` is invalid if any child control is invalid, up to the root `FormGroup`. So binding `[disabled]="form.invalid"` on submit reflects the entire nested tree.

The subtlety with **dynamic/nested** forms is making sure: - Hidden fields don't count (Formly removes their controls — handled). - Newly added `fieldArray` entries register their validators immediately. - Custom async/visa-specific rules surface their errors at the **right nesting level** so the attorney sees which applicant/section is invalid, not just "form invalid."

In EasyVisa. I relied on Formly's nested `FormGroup` / `FormArray` aggregation for the plumbing and wrote **custom validators** for visa-specific rules (date ordering, identifier formats, cross-field consistency). Then I surfaced errors at the correct level — an error message rendered inside the specific applicant's card, while the top-level submit stayed disabled until the whole tree was valid.

```
// a custom validator wired into a Formly field
export function dateRangeValidator(control: AbstractControl): ValidationErrors | null {
  const { fromDate, toDate } = control.value ?? {};
  return fromDate && toDate && new Date(fromDate) > new Date(toDate)
    ? { dateRange: true } : null;
}

{ key: 'addressHistory', type: 'repeat',
  fieldArray: { validators: { validation: [dateRangeValidator] }, fieldGroup: [ /* ...
*/ ] } }
```

```
<button type="submit" [disabled]="form.invalid">Submit case</button>
```

One-liner to land it: "Because it's reactive forms underneath, validity bubbles from every nested group and array up to the root, so a single `form.invalid` binding correctly gates a 200-field, conditionally-shown, repeatable form."

Q8. You wrote custom Formly field types — why and how?

Answer. Formly ships basic types, but real apps need custom ones. A custom type is just an Angular component that extends `FieldType` and renders `field` / `formControl`. Registering it makes `type: 'my-type'` usable in any config.

In EasyVisa. My most valuable one was the `repeat` type powering every repeatable section — a consistent add/remove UI for applicants, address history, dependents, etc. I also wrapped specialized inputs (date pickers, the document/photo control tied to the cropper) as Formly types so forms could reference them by name:

```
@Component({
  selector: 'formly-repeat-section',
  template: `
    <div *ngFor="let f of field.fieldGroup; let i = index" class="repeat-row">
      <formly-field [field]="f"></formly-field>
      <button type="button" (click)="remove(i)">Remove</button>
    </div>
    <button type="button" (click)="add()">Add {{ field.props?.addLabel }}</button>
  `
})
export class RepeatSectionType extends FieldArrayType {}
```

```
// registered once, reused everywhere
FormlyModule.forRoot({
  types: [
    { name: 'repeat', component: RepeatSectionType },
    { name: 'datepicker', component: DatePickerType },
  ],
  validators: [{ name: 'dateRange', validation: dateRangeValidator }],
})
```

The payoff: a whole new repeatable, validated section was **JSON referencing existing types** — that's the reuse story that made new visa forms fast to build.

Q9. How did you manage the form's state, drafts, and prefill?

Answer. With reactive/Formly forms the `model` object is the single source of truth. Prefill = set the model. Save draft = serialize the model. Restore = set the model back. No manual field-by-field wiring.

In EasyVisa. I kept the Formly `model` as one state object, so: - **Prefill** an existing case = load the case JSON into `model`. - **Save draft** = persist the `model` (attorneys filled these over multiple sessions). - **Restore** = hydrate `model` and Formly rebuilt the exact form state, including repeated sections.

```
saveDraft(): void {
  this.caseService.saveDraft(this.caseId, this.model) // model IS the state
    .pipe(takeUntil(this.destroy$)).subscribe();
}
```

That "the model is the state" property is a direct benefit of choosing reactive forms + Formly, and it made a genuinely useful feature (multi-session draft editing) almost free.

Q10. If the requirements/forms changed, how disruptive was it?

Answer. That's exactly what the design optimized for. Because forms were JSON config and conditional logic was `expressions`, most "changes" — add a field, make one conditionally required, add a new repeatable section — were **config edits reusing existing field types**, not new components. That kept mid-sprint form changes cheap.

In EasyVisa. When to change requirements, my honest "what I'd do differently" is I'd have the **form JSON served from the backend** rather than shipped in the frontend build — then non-developers could adjust forms without a release. We shipped configs in the app; making them backend-driven was the logical next step and would've fully realized the "forms as data" vision.

Rapid-fire recap

Concept	Formly mechanism	EasyVisa use
JSON-driven form	<code>FormlyFieldConfig[]</code>	every visa form rendered from config
Nesting	<code>fieldGroup</code> (→ nested <code>FormGroup</code>)	applicant → address / employment
Repeatable	<code>fieldArray</code> (→ <code>FormArray</code>)	N applicants, each with N addresses (nested repeat)
Conditional show/hide	<code>expressions: { hide }</code>	employer fields shown only if employed
Dynamic required	<code>expressions: { 'props.required' }</code>	fields required only when visible

Concept	Formly mechanism	EasyVisa use
Validation bubbling	reactive-forms aggregation	<code>form.invalid</code> gates the submit button
Custom rules	custom validators	date ranges, identifier formats
Reuse	custom field types (<code>repeat</code> , pickers)	new section = JSON, not new code
State/drafts	<code>model</code> = single source of truth	prefill, multi-session draft save/restore
Foundation	reactive forms (not template-driven)	typed, testable, scales to nesting

3. Backend (Grails / Groovy) — Interview Q&A tied to EasyVisa

Honest framing (say this): I was the **frontend engineer**. I owned the Angular side and **consumed** the Grails/Groovy REST APIs — I read and understood the backend, coordinated the API contracts, and occasionally worked in it, but I did not write Grails day-to-day. So my answers are *"here's what Grails is, here's how our backend was shaped, and here's exactly how I consumed it,"* not "I'm a Grails expert." That honesty is a feature, not a weakness — it shows I understood the whole system and knew my boundary.

Stack: Grails (Groovy) backend, GORM over Hibernate to **PostgreSQL**, plus **Neo4j** for graph relationships (see [04-database-postgres-neo4j.md](#)). REST/JSON endpoints consumed by the Angular app.

Q1. What is Grails?

Answer. Grails is a **full-stack web framework built on Groovy** (a JVM language), following **convention over configuration** — the Rails philosophy on the JVM. Under the hood it's **built on Spring Boot and Hibernate**, so you get Spring's dependency injection and Hibernate's ORM, but with far less boilerplate because Grails wires sensible defaults by convention.

Key ideas: - **Groovy** — a dynamic (and optionally typed) JVM language; less ceremony than Java, interoperates with Java libraries. - **Convention over configuration** — put a controller in `grails-app/controllers`, name it `CaseController`, and it's auto-mapped to `/case` routes without XML/annotation wiring. - **GORM** — Grails' Object Relational Mapping, its data-access layer (over Hibernate for SQL, with a Neo4j implementation too). - **Built on Spring** — services are Spring beans, injected by name.

In EasyVisa. The backend was Grails/Groovy exposing REST endpoints. From my seat, the important thing was that it produced clean JSON resources — cases, applicants, petitioners, documents — that my Angular services consumed.

Q2. What is GORM and how does persistence work?

Answer. **GORM** is Grails' ORM. You define **domain classes** (like `Case`, `Applicant`) and GORM maps them to database tables and gives you dynamic finders and CRUD without writing SQL. It's Hibernate-backed for relational stores.

A domain class looks like:

```
class VisaCase {
    String caseNumber
    String status
    Date filedOn

    static hasMany = [applicants: Applicant, documents: Document] // relationships
}
```

```

static constraints = {
    caseNumber blank: false, unique: true
    status inList: ['DRAFT', 'FILED', 'APPROVED', 'REJECTED']
}
}

```

GORM then gives you dynamic finders and CRUD for free:

```

VisaCase.findByCaseNumber('EV-2023-001')
VisaCase.findAllByStatus('FILED')
def c = new VisaCase(caseNumber: 'EV-2023-002', status: 'DRAFT').save()

```

In EasyVisa (honest depth). I understood the domain model — cases have many applicants/ documents, applicants link to petitioners — because I had to consume it and agree on the JSON shape. The `hasMany` /relationship modeling on the Postgres side is what my nested Formly forms mirrored on the frontend. I wasn't authoring GORM constraints daily, but I knew what came from Postgres via GORM versus what came from Neo4j.

Q3. How do controllers and REST endpoints work in Grails?

Answer. A **controller** is a Groovy class in `grails-app/controllers` whose action methods handle requests. Grails maps them by convention (`CaseController` → `/case`), and you can expose REST resources with `@Resource` or explicit URL mappings. Actions render JSON with `respond` or `render ... as JSON`.

```

class CaseController {
    CaseService caseService // Spring injects the service by name

    def show(Long id) {
        def visaCase = caseService.getCase(id)
        respond visaCase // content-negotiated → JSON for our Angular client
    }

    def save() {
        def visaCase = caseService.create(request.JSON)
        respond visaCase, status: CREATED
    }
}

```

Business logic lived in **services** (`grails-app/services`), which are transactional Spring beans injected into controllers — the same layering idea as Spring (controller → service → domain/repository).

In EasyVisa. The controllers produced the REST resources my Angular feature services called — `GET /api/cases/:id`, `POST /api/documents`, etc. My job was agreeing the request/response contract with the backend devs and consuming it.

Q4. How did the Angular frontend consume these APIs?

Answer. Through Angular **feature services** wrapping `HttpClient`, returning typed Observables, with a shared **HTTP interceptor** for auth headers and global error handling. This is the part I owned fully.

In EasyVisa:

```
@Injectable({ providedIn: 'root' })
export class CaseService {
  constructor(private http: HttpClient) {}

  getCase(id: string): Observable<VisaCase> {
    return this.http.get<VisaCase>(`/api/cases/${id}`).pipe(
      catchError(err => { this.notify(err); return EMPTY; })
    );
  }

  saveDraft(id: string, model: unknown): Observable<void> {
    return this.http.put<void>(`/api/cases/${id}/draft`, model);
  }
}
```

- **Typed models** — TS interfaces mirroring the Grails JSON, so the compiler caught contract drift.
- **Interceptor** — attached the auth token and centralized 401/global error handling so components never saw raw HTTP errors.
- **Uploads** — `POST /api/documents` with `FormData` + `reportProgress` (see `01-frontend.md`).

Coordinating these contracts with the Grails team was a big part of my work — we agreed field names, status codes, and error shapes up front so frontend and backend moved in parallel.

Q5. Groovy vs Java — what's different?

Answer (honest, high level). Groovy is a **JVM language** that's more concise and dynamic than Java: optional typing, no boilerplate getters/setters, closures, built-in collection sugar, GStrings (`"Hello $name"`), and easy JSON handling. It's Java-compatible — Groovy can call Java libraries and vice versa. Grails leans on Groovy's dynamic nature for its dynamic finders and conventions.

```
// Groovy — concise
def names = cases.findAll { it.status == 'FILED' }.collect { it.caseNumber }
```

In EasyVisa. I could read Groovy comfortably and understood the concise idioms in the controllers/services, which mattered when debugging an API mismatch. I'd be honest in an interview that Java/Groovy backend wasn't my primary lane — my depth is Angular/TypeScript — but I understood the code well enough to consume it correctly and to reason about where a bug lived (frontend vs API vs data).

Q6. How was global exception / error handling done, end to end?

Answer. Two layers: - **Backend (Grails):** services threw domain exceptions; the framework mapped them to appropriate HTTP status codes with a JSON error body (`{ message, code }`). Grails supports centralized error handling via `URLMappings` error controllers / exception handlers, analogous to Spring's `@ControllerAdvice` . - **Frontend (Angular) — my part:** a shared **HTTP interceptor** caught failures globally, and per-call `catchError` mapped errors to user-friendly messages so a raw 4xx/5xx never reached a component.

In EasyVisa. From the frontend, the contract that mattered was: errors come back as JSON with a `message` I can show. My interceptor handled auth/session-expiry globally, and services `catchError` 'd to a notification service for consistent toasts:

```
// consistent user-facing errors, raw HTTP never reaches the component
uploadDocument(file: File, caseId: string): Observable<UploadState> {
  return this.http.post<UploadState>('/api/documents', this.toFormData(file, caseId))
    .pipe(catchError(err => of({ status: 'error', message: err.error?.message ?? 'Upload
failed' })));
}
```

Q7. Interface vs abstract class (they ask this in a backend/OO context)

Answer. - **Interface** — a pure **contract**: method signatures, no implementation (default methods aside), no instance state. A class can implement **many**. Use it to say "this type can do X." - **Abstract class** — a partial implementation: it can have **implemented methods and state**, and defines a base for subclasses. A class extends **one**. Use it to share common behavior among related types.

Rule of thumb: **interface for a capability/contract across unrelated types; abstract class for shared behavior among related types.**

In EasyVisa (TypeScript angle). On the frontend I used **TS interfaces** constantly to type the Grails JSON contracts and the Formly configs — e.g. `interface VisaCase` , `interface FormlyFieldConfig` -shaped models — because they're compile-time contracts with zero runtime cost. Where I had shared component behavior (base upload panel logic), a base **class** was the right tool. That's the exact interface-vs-class distinction, applied in my real code.

Q8. try / catch / finally — and the RxJS equivalent

Answer. - `try` — wrap code that might throw. - `catch` — handle the exception (log, remap, recover). - `finally` — always runs, thrown or not — for cleanup (close resources, stop a spinner).

```
try {
  const parsed = JSON.parse(rawDraft);
  this.model = parsed;
} catch (e) {
  this.notify('Could not load saved draft');
```

```

} finally {
  this.loading = false;    // always stop the spinner
}

```

In **EasyVisa**. In async/observable code the equivalent is `catchError` (the catch) and `finalize` (the finally):

```

this.caseService.getCase(id).pipe(
  catchError(err => { this.notify(err); return EMPTY; }), // catch
  finalize(() => this.loading = false)                  // finally – always runs
).subscribe(c => this.case = c);

```

I used `try/catch/finally` for synchronous risky code (parsing saved draft JSON) and `catchError` / `finalize` for the HTTP/observable flows, which is the more common case in an Angular app.

Rapid-fire recap

Topic	EasyVisa reality
What is Grails	Groovy, convention-over-config, built on Spring + Hibernate
GORM	domain classes → tables, dynamic finders, <code>hasMany</code> relationships (Postgres)
Controllers	Groovy classes, convention-mapped, <code>respond</code> JSON, service layer
How I consumed it	Angular feature services + <code>HttpClient</code> , typed models, interceptor
Contract work	agreed field names/status codes/error shapes with the Grails team
Groovy vs Java	concise, dynamic JVM language; I read it, consumed it, not my main lane
Error handling	backend JSON <code>{message}</code> + frontend interceptor/ <code>catchError</code>
Interface vs abstract	contract vs shared behavior; I used TS interfaces for API/Formly types
try/catch/finally	sync risky code; <code>catchError</code> / <code>finalize</code> for observables
My honest boundary	owned the Angular frontend; consumed the Grails/Neo4j backend

4. Databases (PostgreSQL + Neo4j) — Interview Q&A tied to EasyVisa

Honest framing: I was frontend-focused. I understood the **data architecture** and consumed data that came from both stores through the Grails REST APIs. I did not write Cypher or tune Postgres day-to-day — so my answers explain *why we had two databases, what each was for, and how the shapes showed up in the UI I built*. Knowing the "why" of a polyglot-persistence design is exactly the system-understanding interviewers want.

Stack: **PostgreSQL** for relational/transactional data (via GORM/Hibernate) + **Neo4j** graph database for the relationships between applicants, petitioners, and cases.

Q1. Why two databases? (the whole point — nail this)

Answer. This is **polyglot persistence** — use the right store for the shape of the data:

- **PostgreSQL** held the **core relational, transactional data**: case records, applicant details, document metadata, statuses. This data needs **ACID transactions, integrity constraints, and reliable updates** — a relational DB is exactly right for it.
- **Neo4j** (a **graph database**) modeled the **relationships**: applicants ↔ petitioners ↔ cases ↔ employers/family links. Visa cases are fundamentally a **graph of connected entities**, and questions like "who is connected to this case, and how, several hops out?" are **native in a graph** and **painful with deep relational joins**.

The one-liner: *Postgres for the records and transactions; Neo4j for the web of relationships. We stored each kind of data where the queries were natural.*

Q2. What is a graph database? Nodes and relationships?

Answer. A graph DB stores data as **nodes** (entities) and **relationships/edges** (the connections between them), where **relationships are first-class citizens** — they're stored directly, with their own type and properties, not derived at query time from foreign keys.

- **Node** — an entity, e.g. an **Applicant**, **Petitioner**, or **Case**, with properties (name, id).
- **Relationship** — a typed, directed edge, e.g. **(:Applicant)-[:APPLIES_FOR]->(:Case)** or **(:Petitioner)-[:SPONSORS]->(:Applicant)**, and edges can have properties too (e.g. a **since** date).

Because connections are stored as real edges, **traversing** them ("follow this applicant's links") is a pointer hop, not a join — so multi-hop relationship queries stay fast even as depth grows.

In EasyVisa. The nodes were applicants, petitioners, cases (and related parties like employers/family), and the edges captured how they connected. That's the natural shape of immigration data — a case isn't one row, it's a small network of people and roles.

Q3. Show me some Cypher. What does a query look like?

Answer. Cypher is Neo4j's query language, and it's **visual** — you draw the pattern in ASCII art: `(node)-[:REL]->(node)`.

Find everyone connected to a given case:

```
MATCH (c:Case { caseNumber: 'EV-2023-001' })<-[r]-(person)
RETURN person, type(r)
```

Traverse further — who is connected to this case's applicants, two hops out (family, petitioners, employers):

```
MATCH (c:Case { caseNumber: 'EV-2023-001' })<-[:APPLIES_FOR]-(a:Applicant)
MATCH (a)-[rel]-(related)
RETURN a, type(rel), related
```

Find shared petitioners across cases (a fraud/relationship-insight style query):

```
MATCH (p:Petitioner)-[:SPONSORS]->(:Applicant)-[:APPLIES_FOR]->(c:Case)
RETURN p, collect(c.caseNumber) AS cases
```

In EasyVisa (honest). Cypher lived on the Grails backend — I didn't author it day-to-day. But I understood what these traversals returned because I **rendered them**: the "related parties on a case" views I built in Angular were backed by Neo4j traversals on the server. So I knew the shape of the graph data coming down and modeled my UI around it.

Q4. Why is "who's connected to this case" painful in SQL but natural in a graph?

Answer. In a relational model, relationships are **implicit** — encoded as foreign keys — so answering "everyone connected to this case, N hops out" means **repeated JOINS** (often self-joins through junction tables), one JOIN per hop. As the depth grows, the SQL gets complex and slow, and **variable-depth** ("as far as the connections go") is genuinely hard to express in standard SQL.

In a graph, relationships are **explicit edges**, so the same question is a **traversal**: start at the case node and follow edges. Variable depth is trivial (`-[:REL*1..3]-`), and performance depends on how many nodes you actually touch, not on the total table size.

```
-- Relational: 2 hops already needs multiple joins through junction tables
SELECT p.*
FROM cases c
JOIN case_applicants ca ON ca.case_id = c.id
JOIN applicants a       ON a.id = ca.applicant_id
JOIN applicant_petitioner ap ON ap.applicant_id = a.id
```

```
JOIN petitioners p      ON p.id = ap.petitioner_id
WHERE c.case_number = 'EV-2023-001';
```

```
-- Graph: the same idea, plus arbitrary depth for free
MATCH (c:Case { caseNumber: 'EV-2023-001' })-[*1..3]-(connected)
RETURN DISTINCT connected
```

In EasyVisa. The "related parties" and connection views were the reason Neo4j existed. Expressing them as ever-deeper joins would have been slow and unmaintainable; as graph traversals they were natural. That's the concrete justification for the two-database design.

Q5. When would you choose a graph DB vs a relational DB?

Answer. - Relational (Postgres): structured, tabular data; strong transactional integrity; aggregations and reporting; data where relationships are shallow and known. → **case records, statuses, document metadata, anything needing ACID.** - **Graph (Neo4j):** data where the **relationships are the value** and you traverse them deeply or at variable depth — social networks, fraud rings, recommendations, and exactly this: **networks of people connected to cases.**

The mature answer is **polyglot persistence**: it's not either/or. EasyVisa used **both**, storing each kind of data where its queries were natural — transactional records in Postgres, the relationship web in Neo4j.

In EasyVisa. If I were asked to defend it: putting relationships in Neo4j didn't mean abandoning Postgres — the authoritative transactional records stayed relational for integrity, and Neo4j gave us fast, expressive relationship queries. Right tool per job.

Q6. How did the two stores stay consistent, and how did you consume them?

Answer (honest). The **backend owned the coordination** — the Grails services wrote to Postgres for the record data and maintained the corresponding graph in Neo4j so relationship queries reflected the current records. From the **frontend**, I didn't see two databases; I saw **REST endpoints**: case/applicant/document data (Postgres-backed) and "related parties / connections" data (Neo4j-backed) came through the same API surface as JSON.

In EasyVisa. My job was to know **which data came from where** so I coordinated the right contracts with the backend team — record-shaped data (forms, statuses, documents) versus relationship-shaped data (the connection graph views). I consumed both as typed Observables and rendered them; the dual-write consistency was a server-side concern I understood but didn't own.

Rapid-fire recap

Question	Answer
Why two DBs	polyglot persistence — Postgres for records/transactions, Neo4j for relationships

Question	Answer
Postgres role	ACID transactional data: cases, applicants, documents, statuses
Neo4j role	the graph of applicants ↔ petitioners ↔ cases ↔ related parties
Graph DB basics	nodes + first-class relationship edges; traversal not joins
Cypher	<code>MATCH (a) -[:REL]->(b) RETURN ...</code> — pattern matching in ASCII art
SQL pain	N hops = N joins; variable depth is hard; graph traversal is native
Graph vs relational	relationships-are-the-value & deep traversal → graph; integrity/tabular → relational
My honest role	frontend; understood the architecture, consumed both via REST, knew which data came from where

5. Behavioral, Pitch & STAR Stories — EasyVisa

Purpose: the human/story layer. The interviewers fail me on "project understanding," so the fix is to make every behavioral answer concrete — a real EasyVisa feature, a real decision, a real trade-off. Use STAR: **S**ituation, **T**ask, **A**ction, **R**esult.

The 60-second project pitch (memorize)

"EasyVisa is a visa case management platform for immigration attorneys — it manages applications, documents, and the relationships between cases, applicants, and petitioners. I was the frontend engineer, from late 2021 to early 2024, on a team of about six to eight. The app was Angular, talking to a Grails/Groovy backend, with PostgreSQL for transactional data and Neo4j as a graph database for the relationships between parties.

I owned two things end-to-end. First, the **document portal** — accordion upload panels where attorneys upload the many documents a case needs, with per-file progress and isolated error handling using Angular's `HttpClient` event stream. Second — and this is the part I'm proudest of — the **dynamic forms**. Visa forms are enormous, conditional, and repeatable: a case can have many applicants, each with nested address and employment histories, and fields appear or disappear based on earlier answers. Instead of hand-coding each form, I built them with **Angular Formly, driven by JSON configuration** — so new forms and new conditional rules became *configuration, not code*. I also customized an image cropper for applicant photos, drove an Angular version upgrade, built reusable components, and wrote the unit tests and documentation. My work made complex, document-heavy visa workflows genuinely manageable for attorneys."

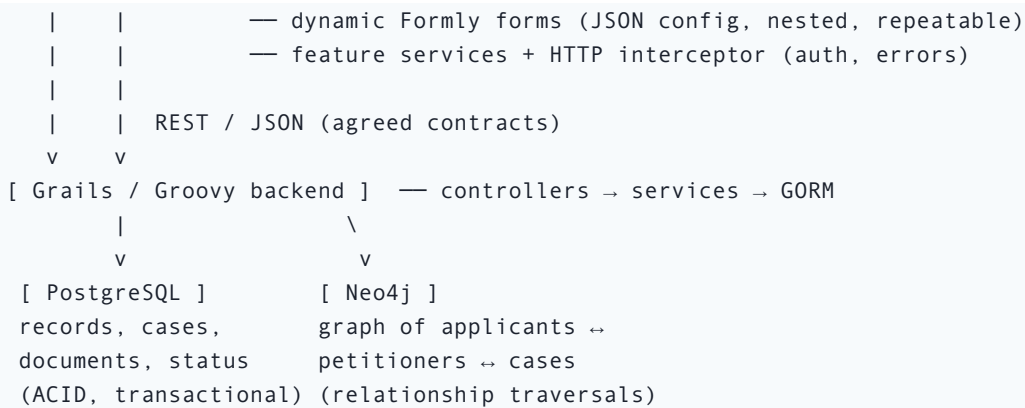
My role (crisp version)

"Frontend engineer owning the document portal and the dynamic Formly forms, plus shared/reusable components. I coordinated the REST API contracts with the Grails/Groovy backend team, consumed data from both Postgres and Neo4j through those APIs, wrote Jasmine/Karma unit tests for all my components and services, and documented specs and user manuals. I was frontend-focused — I understood the whole system but my hands-on ownership was the Angular side."

The architecture to draw (whiteboard walkthrough)

Draw this left-to-right and narrate it:

```
[ Attorney browser ]
      |
      v
[ Angular SPA ] — document portal (upload panels, HttpClient progress)
```



Narration: "Attorneys use an Angular SPA. My two big modules are the document portal and the Formly-driven forms. The app talks to a Grails/Groovy backend over REST — controllers to services to GORM. Transactional records live in Postgres; the relationships between people and cases live in Neo4j because that's a graph problem. I owned the Angular layer and the API contracts."

STAR story 1 — The Formly forms decision (my signature story)

- **S (Situation):** Visa forms are huge, conditional, and repeatable — a case can have N applicants, each with nested address/employment history, and fields appear/disappear based on earlier answers. The initial instinct on the team was to hand-code each form.
- **T (Task):** Build the forms so they were maintainable and could change as immigration requirements changed — without a code release and a new component for every form or rule.
- **A (Action):** I introduced **Angular Formly** and drove a **JSON-driven** approach: forms rendered from a `FormlyFieldConfig` array, `fieldGroup` for nesting, `fieldArray` for repeatable sections (including nested repeats — applicants containing address histories), and `expressions` for conditional show/hide and dynamic `required`. I wrote reusable custom field types (a `repeat` section type used everywhere) and custom validators for visa-specific rules, and I relied on reactive-forms validity aggregation so errors bubbled up and the submit button gated the whole nested tree.
- **R (Result):** New forms and new conditional rules became **configuration, not code** — dramatically faster to add and change, and far less bug-prone. Mid-sprint form changes went from "new component + tests" to "edit JSON." It was my biggest technical decision on the project and it clearly paid off at that scale of forms.
- **Trade-off I volunteer:** steeper learning curve and some complexity writing custom field types up front — worth it for the long-term savings; over-engineering if there'd only been two simple forms.

STAR story 2 — The document upload challenge

- **S:** Attorneys upload many documents per case, some large. Naive uploads blocked the UI, risked timeouts, and if one file failed it could take down the whole batch's UX.
- **T:** Build an upload experience with per-file progress, isolated error handling, and no UI freezing.
- **A:** I used `HttpClient` with `reportProgress: true` and `observe: 'events'`, reading `HttpEventType.UploadProgress` to drive a per-file progress bar and `HttpEventType.Response` to

mark completion. I organized uploads into **accordion panels by document type**, each with its **own progress/error state**, so one failed upload showed its error while the others kept going. Uploads were concurrent but throttled, and every subscription used `takeUntil(destroy$)` so navigating away mid-upload didn't leak.

- **R:** Attorneys got responsive, transparent uploads with clear per-file feedback, and a single failure no longer sank the case. The upload panel became a reusable component driven by a document-type config.
- **What I'd do differently:** for very large files I'd move to **chunked/resumable** uploads from the start — I flagged it as the next iteration; we shipped the progress-stream version first because it covered the real file sizes in use.

STAR story 3 — Customizing the image cropper

- **S:** Certain documents — applicant photos especially — had strict dimension, aspect-ratio, and format requirements. Attorneys shouldn't have to pre-edit images in another tool.
- **T:** Enforce those photo constraints in-app and feed compliant images into the upload flow.
- **A:** I wrapped and **customized an Angular image-cropper library**: locked the crop box to the required aspect ratio, forced the output size/format, and **post-processed the output blob** before it entered the document upload pipeline. I wrapped it as a reusable component (and a Formly field type) so it could drop in anywhere.
- **R:** Photos were compliant by construction — no back-and-forth, no rejected documents for wrong dimensions — and the cropper became a reusable control across the app.

STAR story 4 — A debugging story

- **S:** A conditional field in a nested, repeatable Formly section wasn't behaving — after adding/removing applicants, a field that should have been hidden (or its `required` rule) wasn't re-evaluating correctly, and the submit button's enabled state didn't match what the attorney saw.
- **T:** Find whether the bug was in my form config, in Formly's rebuild of the nested `FormArray`, or in a stale validation state — and fix it at the right layer.
- **A:** I reproduced it with exact steps, then isolated the layer: I inspected the underlying `FormGroup` / `FormArray` validity and the model as sections were added/removed, and traced it to how the conditional `expressions` and validators re-registered when the array rebuilt. I fixed it so hidden fields dropped their controls from validation and newly added entries registered validators immediately, then added a **unit test** asserting the show/hide and `required` behavior across add/remove so it couldn't silently regress.
- **R:** Submit state matched the visible form exactly, and the regression test locked it in. This is why I prioritized unit tests on the conditional logic — that's precisely where nested-dynamic-form bugs hide.
- **Method I'd emphasize:** reproduce first, isolate the layer (frontend vs API vs data via network tab and logs), fix at the correct layer, add a regression test — I avoid guess-and-deploy.

"What would you do differently on EasyVisa?"

"Three things. One, move large uploads to **chunked/resumable** from the start rather than as a follow-up. Two, serve the **form JSON configs from the backend** instead of shipping them in the frontend build — then non-developers could adjust forms without a release, fully realizing the 'forms as data' idea. Three, add more **integration tests around the conditional form logic**, since that's where regressions hide. I'd also move the heavy document-portal components to **OnPush** change detection for a cleaner perf story."

Collaboration, Agile & Git

Agile process:

"Two-week sprints — planning, daily standups, sprint review/demo, retro. Work tracked in Jira as stories and bugs, code in Git with feature branches and PR reviews. Standup is where I flagged blockers early, especially anything waiting on a backend API contract."

Cross-team collaboration:

"The tightest coordination was with the Grails/Groovy team on the **REST contracts** — agreeing field names, status codes, and error shapes up front so frontend and backend moved in parallel — and on **what data came from Postgres vs Neo4j** so I knew which endpoint served record data versus relationship data."

Code reviews:

"I look for correctness first, then maintainability: clear naming, proper unsubscribe/async-pipe usage, error and loading states, no obvious perf traps, and adequate tests. Specific, kind, actionable comments, and I distinguish blocking issues from nits."

Handling changing requirements:

"I clarify the change with the PM, assess impact on the sprint commitment, and raise it transparently rather than silently absorbing it. And on EasyVisa the config-driven Formly design made many 'changes' cheap — a new field or rule was JSON, not a new component."

Biggest technical decision + trade-off:

"Choosing Angular Formly with JSON-driven config over hand-coding each visa form. The trade-off was a steeper learning curve and custom-field-type complexity versus huge long-term savings — forms and conditional logic became configuration. At that scale of forms, it clearly paid off."

Disagreement handled well:

"When we debated hand-coding forms versus a config-driven approach, rather than just asserting, I showed the maintenance cost concretely — how many conditional, repeatable forms we'd have to build and re-build by hand — and how Formly turned each new rule into

one line of config. We aligned on Formly, and I committed to owning the custom field types that made it work."

Questions I can ask the interviewer (shows engagement)

- "How do you handle dynamic/complex forms — Formly, hand-coded, or a schema service?"
 - "Do you use polyglot persistence like we did with Postgres + Neo4j, or a single store?"
 - "What's your change-detection and performance strategy on large Angular trees?"
 - "How is the frontend/backend API contract agreed and versioned on your team?"
-

Final confidence anchors

- Lead with the **Formly JSON-driven forms** — it's my strongest, most unique story.
- Always land technical answers with **"In EasyVisa, I..."**.
- Be honest about the **frontend boundary** (consumed Grails/Neo4j) — it reads as senior, not weak.
- Own the **trade-offs** (chunked uploads, backend-served form JSON, OnPush) — owning what I'd improve is what separates 5-years from junior.